

LEC28 - Wrap Up

Recursion Tips

- Each time we call a recursive method
 - Everything it needs should be sent through parameters
 - Everything it must report back should come through parameters
 - Don't attempt to work around this protocol by using local variables
 - Each call has its own stack frame with its own instance of local variables
 - Therefore, nothing can accumulate them across calls
-

Helper Methods

- Sometimes, a methods API does "have enough" to support the recursion
 - A helper may have an additional parameter
 - Ex. The helper method for `Tree.__str__()`
 - Adds a parameter for indent
 - A helper method can have an additional return value
 - Ex. The helper for `Tree.average()`
 - Returns a tuple with total, and number of items
-

Documentation

- Write good docstrings for good communication
 - Communicate what your code does and how it is intended to be used
 - What is assumed and implied
 - What is the expected result
 - NOT how the code achieves the result internally
 - Remember interface vs implementation
-

Debugging

Main Idea

- Follow the logic of your code, step by step
 - Analyze if the behaviour of your code is as expected
- Debugging is an important skill
 - Particularly for complex code
 - Gets you out of tricky spots
 - Better alternative to using print statements (logging)
 - Logging does still have its benefits though

Tracing Recursive Functions

- An easy way to verify is your implementation exhibits the intended behaviour
 - As with any debugging, find bugs or corner cases that are not addressed
 - Still need to draw diagrams
-

Testing

- Important to test as you go
- Write meaningful tests
 - Very important skill
 - Consider corner cases
 - Your code must gracefully handle all exceptional cases instead of crashing
 - Remember precondition and representation invariants
- Retest your code on every revision
 - Whenever you make substantial changes, retest that everything that used to work still does